

미지의 던전 탐험기

Java 콘솔 기반 텍스트 어드벤처 게임 프로젝트 발표

과목명

자바 프로그래밍

발표자

최동현

프로그램 개요

모험가가 마을에서 선택하고, 던전에서 랜덤 이벤트를 경험하는 콘솔 RPG 게임

게임 핵심 구조

- 시작 상태: HP 100, 골드 0, 포션 1개
- 마을 메뉴: 던전 / 잡화점 / 여관 선택
- 던전 이벤트: 몬스터, 보물, 함정 중 하나 발생
- 전투 선택: 싸운다, 포션 마시기, 도망친다
- HP가 0 이하가 되면 GAME OVER 출력

주요 언어

Java

입력 방식

Scanner

난수 처리

Random

```

> ~/De/C/E/AdventureGame/bin java AdventureGame
=====
[ 미지의 던전 탐험기 ]
=====

-----
[ 현재 상태 ] HP: 100 | 골드 : 0 | 포션 : 1개
-----

당신은 평화로운 마을 광장에 있습니다.
어디로 이동하시겠습니까?

-----
1. 어두운 던전으로 향한다.
2. 잡화점에서 포션을 산다. (비용 : 30골드)
3. 여관에서 휴식한다. (비용 : 20골드, HP: 50회복)
▶ 당신의 선택 (1-3): █
  
```

실행 화면: 게임 시작 후 마을 메뉴 출력

전체 코드 구성 한눈에 보기

전역 상태 변수

hp 플레이어 체력

gold 보유 골드

potions 포션 개수

Scanner 사용자 입력

Random 난수 이벤트

main()

게임 시작
while 반복

showTown()

상태 출력
메뉴 분기

enterDungeon()

랜덤 이벤트
50/30/20

encounterMonster()

전투 루프
3가지 행동

buyPotion()

30G 소비
포션 +1

restAtInn()

20G 소비
HP +50

전체 코드 — 클래스 시작 · main() · 마을 메뉴

```

1 import java.util.Scanner;
2 import java.util.Random;
3
4 public class AdventureGame {
5     // 플레이어 상태 전역 변수
6     static int hp = 100;
7     static int gold = 0;
8     static int potions = 1;
9
10    static Scanner sc = new Scanner(System.in);
11    static Random rand = new Random();
12
13    public static void main(String[] args) {
14        System.out.println("=====");
15        System.out.println("  [미지의 던전 탐험기]  ");
16        System.out.println("=====");
17
18        // HP가 0 이하면 게임 종료
19        while (hp > 0) {
20            showTown(); // 마을 함수 호출
21        }
22
23        System.out.println("\n[사유] 당신의 HP가 0이 되었습니다.");
24        System.out.println("당신의 모험은 여기서 끝이 납니다. (GAME OVER)");
25    }
26
27    // 마을
28    static void showTown() {
29        System.out.println("\n-----");
30        System.out.println("[ 현재 상태 ] HP: " + hp + " | 골드: " + gold + " | 포션: " + potions + "개");
31        System.out.println("-----");
32        System.out.println("당신을 평화로운 마을 광장에 있습니다.");
33        System.out.println("어디로 이동하시겠습니까?");
34        System.out.println("-----");
35        System.out.println("1. 어두운 던전으로 향한다.");
36        System.out.println("2. 잔화점에서 포션을 산다. (비용: 30골드)");
37        System.out.println("3. 여관에서 휴식한다. (비용: 20골드, HP: 50회복)");
38        System.out.print("▶ 당신의 선택 (1-3): ");
39
40        int choice = sc.nextInt(); // 선택 입력
41
42        if (choice == 1) {
43            enterDungeon(); // 던전 함수 호출

```

전체 코드 — showTown() 분기 · 던전 이벤트

```

44 |     } else if (choice == 2) {
45 |         buyPotion(); // 잔화점 함수 호출
46 |     } else if (choice == 3) {
47 |         restAtInn(); // 여관 함수 호출
48 |     } else {
49 |         System.out.println("잘못된 선택입니다. 다시 선택해주세요.");
50 |     }
51 | }
52 |
53 | // 던전 진입
54 | static void enterDungeon() {
55 |     System.out.println("-----");
56 |     System.out.println("\n( ... 던전으로 진입합니다 ... )");
57 |     System.out.println("안쪽에서 몬스터의 소리가 들립니다.");
58 |
59 |     int event = rand.nextInt(10); // 0 ~ 9 사이 랜덤 숫자 생성
60 |
61 |     if (event < 5) { // 50% 확률로 몬스터 조우
62 |         encounterMonster();
63 |     } else if (event < 8) { // 30% 확률로 보물 발견
64 |         System.out.println("\n[ 이벤트 발생! ]");
65 |         System.out.println("던전 구석에서 낡은 보물상자를 발견했습니다!");
66 |         int findGold = 20 + rand.nextInt(31); // 20~50 골드 획득
67 |         gold += findGold;
68 |         System.out.println(findGold + " 골드를 획득했습니다! (현재 골드: " + gold + ")");
69 |     } else { // 20% 확률로 함정
70 |         System.out.println("\n[ 경고! ]");
71 |         System.out.println("앗! 발을 헛디뎌 함정에 빠졌습니다.");
72 |         hp -= 15; // 체력 15 감소
73 |         System.out.println("체력을 15 잃었습니다. (현재 HP: " + hp + ")");
74 |     }
75 | }
76 |
77 | // 몬스터 전투
78 | static void encounterMonster() {
79 |     System.out.println("\n[ 전투 발생! ]");
80 |     System.out.println("험악하게 생긴 [고블린]이 나타났습니다!");
81 |
82 |     while (true) {

```

전체 코드 — 몬스터 전투 루프

```

83 | System.out.println("\n1. 싸운다 (체력 소모 예상, 골드 획득);");
84 | System.out.println("\n2. 포션 사용하기 (현재 " + potions + "개 보유)");
85 | System.out.println("\n3. 도망친다 (확률적 성공)");
86 | System.out.print("▶ 당신의 선택 (1-3): ");
87 |
88 | int action = sc.nextInt();
89 |
90 | if (action == 1) {
91 |     int damage = 10 + rand.nextInt(21); // 10~30 데미지
92 |     int reward = 30;
93 |     hp -= damage; // 10~30 hp피해를 입고,
94 |     gold += reward; // 30골드를 받음
95 |
96 |     System.out.println("\n치열한 전투 끝에 고블린을 물리쳤습니다!");
97 |     System.out.println(damage + "의 피해를 입었지만, " + reward + " 골드를 얻었습니다.");
98 |     break; // 루프 탈출
99 |
100 | } else if (action == 2) {
101 |     if (potions > 0) { // 포션이 있다면
102 |         potions--; // 포션을 차감하고
103 |         hp += 40; // 40을 회복
104 |         if (hp > 100) hp = 100; // hp 100이 최대
105 |         System.out.println("\n포션을 마셔 체력을 40 회복했습니다! (현재 HP: " + hp + ")");
106 |     } else {
107 |         System.out.println("\n포션이 없습니다!");
108 |     }
109 | } else if (action == 3) {
110 |     if (rand.nextBoolean()) { // boolean으로 0혹은 1로 50%확률로 도망
111 |         System.out.println("\n무사히 마을로 도망쳤습니다.");
112 |         break; // 루프 탈출
113 |     } else {
114 |         System.out.println("\n도망치는데 실패했습니다! 고블린에게 등짝을 맞아 체력을 10 잃었습니다.");
115 |         hp -= 10; // 도망 실패시 hp 10소모
116 |     }
117 | } else {
118 |     System.out.println("잘못된 입력입니다.");
119 | }
120 |
121 | if (hp <= 0) { // hp 0이하가 되면 게임 종료
122 |     return;
123 | }
124 | }
125 | }
126 |

```

전체 코드 — 잡화점 · 여관 · 종료부

```
127 | // 잡화점
128 | static void buyPotion() {
129 |     System.out.println("\n[ 잡화점 ]");
130 |     if (gold >= 30) {
131 |         gold -= 30;
132 |         potions++;
133 |         System.out.println("30골드를 지불하고 포션을 1개 구매했습니다.");
134 |     } else {
135 |         System.out.println("골드가 부족합니다! (필요 골드: 30, 현재 골드: " + gold + ")");
136 |     }
137 | }
138 |
139 | // 여관
140 | static void restAtInn() {
141 |     System.out.println("\n[ 여관 ]");
142 |     if (gold >= 20) {
143 |         gold -= 20;
144 |         hp += 50;
145 |         if (hp > 100) hp = 100; // hp 100 최대
146 |         System.out.println("푹 쉬고 일어났습니다. 체력이 회복되었습니다. (현재 HP: " + hp + ")");
147 |     } else {
148 |         System.out.println("골드가 부족합니다!");
149 |     }
150 | }
151 | }
```

실행 화면 — 게임 시작과 마을 선택

main()에서 배너와 현재 상태를 출력한 뒤 showTown() 메뉴가 표시됩니다.

```

> 🍏 > ~/De/C/E/AdventureGame/bin java AdventureGame
=====
[ 미지의 던전 탐험기 ]
=====

-----
[ 현재 상태 ] HP: 100 | 골드: 0 | 포션: 1개
-----

당신은 평화로운 마을 광장에 있습니다.
어디로 이동하시겠습니까?

-----
1. 어두운 던전으로 향한다.
2. 잡화점에서 포션을 산다. (비용: 30골드)
3. 여관에서 휴식한다. (비용: 20골드, HP: 50회복)
▶ 당신의 선택 (1-3): █
  
```

게임 시작 화면: HP 100, 골드 0, 포션 1개

설명 포인트

- 프로그램 시작 시 제목 배너 출력
- 전역 변수 hp/gold/potions 값 표시
- 1~3번 메뉴로 다음 행동 선택
- 입력은 Scanner.nextInt()로 처리
- 잘못된 값이면 오류 메시지 출력

```

while (hp > 0) {
    showTown();
}

int choice = sc.nextInt();
  
```


실행 화면 — 몬스터 전투와 보상 처리

던전 이벤트가 몬스터로 판정되면 전투 메뉴가 출력되고, 싸우면 피해와 골드 보상이 반영됩니다.

▶ 당신의 선택 (1-3): 1

(... 던전으로 진입합니다 ...)
안쪽에서 몬스터의 소리가 들립니다 .

[전투 발생 !]
험악하게 생긴 [고블린]이 나타났습니다 !

1. 싸운다 (체력 소모 예상 , 골드 획득)
 2. 포션 마시기 (현재 1개 보유)
 3. 도망친다 (확률적 성공)
- ▶ 당신의 선택 (1-3): 1

치열한 전투 끝에 고블린을 물리쳤습니다 !
22의 피해를 입었지만 , 30 골드를 얻었습니다 .

전투 발생 후 1번 싸운다 선택

1. 어두운 던전으로 향한다 .
2. 잡화점에서 포션을 산다 . (비용 : 30골드)
3. 여관에서 휴식한다 . (비용 : 20골드 , HP: 50회복)
▶ 당신의 선택 (1-3): 1

(... 던전으로 진입합니다 ...)
안쪽에서 몬스터의 소리가 들립니다 .

[경고 !]
앗 ! 발을 헛디뎈 함정에 빠졌습니다 .
체력을 15 잃었습니다 . (현재 HP: 63)

함정 발생 예시: HP 15 감소

전투 로직 핵심

- $damage = 10 + rand.nextInt(21) \rightarrow 10 \sim 30$ 피해
- $reward = 30 \rightarrow$ 고블린 처치 보상 고정
- break로 전투 루프 탈출 후 마을로 복귀
- HP가 0 이하가 되면 return으로 전투 종료

실행 화면 — 보물 · 잡화점 · 여관

던전 보상과 마을 시설 이용 결과가 전역 상태 변수에 즉시 반영됩니다.

▶ 당신의 선택 (1-3): 1

(... 던전으로 진입합니다 ...)
안쪽에서 몬스터의 소리가 들립니다.

[이벤트 발생!]
던전 구석에서 낡은 보물상자를 발견했습니다!
39 골드를 획득했습니다! (현재 골드: 69)

보물 발견: 20~50 골드 획득

1. 어두운 던전으로 향한다.
2. 잡화점에서 포션을 산다. (비용: 30골드)
3. 여관에서 휴식한다. (비용: 20골드, HP: 50회복)
▶ 당신의 선택 (1-3): 2

[잡화점]
30골드를 지불하고 포션을 1개 구매했습니다.

잡화점: 30골드 지불 후 포션 구매

1. 어두운 던전으로 향한다.
2. 잡화점에서 포션을 산다. (비용: 30골드)
3. 여관에서 휴식한다. (비용: 20골드, HP: 50회복)
▶ 당신의 선택 (1-3): 3

[여관]
폭 쉬고 일어났습니다. 체력이 회복되었습니다. (현재 HP: 100)

여관: 20골드 지불 후 HP 50 회복

상태 변화 정리

- 보물: gold += 20~50
- 잡화점: gold -= 30, potions++
- 여관: gold -= 20, hp += 50
- HP는 100을 넘지 않도록 제한

핵심 로직 요약

입력

Scanner로 번호를 입력받고 if-else로 메뉴를 분기합니다.

랜덤

Random.nextInt(10)으로 몬스터 50%, 보물 30%, 함정 20%를 구현합니다.

반복

while 루프로 게임과 전투를 반복하고 break/return으로 흐름을 제어합니다.

개선 아이디어

- 몬스터 종류 다양화: 고블린, 오크, 보스 몬스터 등급 분리
- 플레이어 스탯 추가: 공격력, 방어력, 레벨, 경험치 구현
- 파일 입출력 적용: 세이브/로드 기능으로 게임 진행 저장
- 콘솔 시각화 강화: HP 바, 아이템 목록, 전투 로그 정리

정리

1 텍스트 RPG 구조

마을 → 던전 → 결과 반영 → 반복 구조
로 게임 흐름을 구성했습니다.

2 Java 기본 문법 활용

조건문, 반복문, static 변수, Scanner,
Random을 중심으로 구현했습니다.

3 실행 결과 검증

실제 실행 화면을 통해 전투, 보물, 상점,
여관 기능을 확인했습니다.